Let's begin with the foundation: **Fundamental Goals & Metrics**.

Part 1: Fundamental Goals & Metrics

These are the primary requirements you must clarify at the start of any system design interview (e.g., "Is this system read-heavy? Does it need real-time performance?").

1. Scalability

Scalability is not just about "handling more users." It is the system's ability to handle a growing amount of work by adding resources to the system.

A. Vertical Scaling (Scale Up)

- **Concept:** You add more power (CPU, RAM, Storage) to your existing machine.
- **Analogy:** You have a car that can't go fast enough, so you swap the engine for a Ferrari engine. It's still one car, just stronger.
- **Pros:**
    - **Simplicity:** No need to change code or architecture; everything remains on one node.
    - **Inter-process communication:** Fast, because everything is in the same memory space.
- **Cons:**
    - **Hard Limit:** There is a hardware ceiling (you can only buy so much RAM).
    - **Single Point of Failure (SPOF):** If that one monster server crashes, the whole app dies.
    - **Expensive:** High-end hardware follows a curve of diminishing returns.

B. Horizontal Scaling (Scale Out)

- **Concept:** You add more machines to your pool of resources.
- **Analogy:** You have a delivery job too big for one car, so you buy three more cars and hire three drivers.
- **Pros:**
    - **Infinite Scale:** Theoretically, you can keep adding servers endlessly.

    - **Resilience:** If one server dies, the others take over.
- **Cons:**
    - **Complexity:** You now need a Load Balancer to distribute traffic.
    - **Data Consistency:** Keeping data synced across multiple nodes is difficult (CAP theorem comes into play here).
    - **Network Latency:** Communication between servers (RPC/HTTP) is slower than internal memory calls.

Feature	Vertical Scaling (Scale Up)	Horizontal Scaling (Scale Out)
Limit	Hard hardware limit	Theoretically unlimited
Complexity	Low	High (requires load balancing, sharding)
Cost	High (specialized hardware)	Low (commodity hardware)
Failover	Difficult (Single Point of Failure)	Easy (Redundancy)

---

2. Availability

Availability is the percentage of time a system is operational and able to respond to requests. It is often strictly defined in a Service Level Agreement (SLA).

- **Calculation:**

$$Availability = \frac{\text{Uptime}}{\text{Uptime} + \text{Downtime}}$$

- **The "Nines":** In industry, we measure availability by the number of nines.

"Nines"	Availability	Downtime per Year	Example System
Two Nines	99%	3.65 days	Small internal tools
Three Nines	99.9%	8.76 hours	Standard web apps
Four Nines	99.99%	52.6 minutes	Enterprise SaaS
Five Nines	99.999%	5.26 minutes	Telecom, Pacemakers, Banking

How to improve Availability?

- **Redundancy:** If a component fails, another must be ready to replace it immediately.
- **Failover:** The automatic process of switching to a redundant system upon failure (e.g., Active-Passive database setup).

---

3. Latency vs. Throughput

These two are often confused but are distinct performance metrics.

A. Latency (Speed)

- **Definition:** The time it takes for a single packet of data to travel from source to destination.
- **Measured in:** Milliseconds (ms).
- **Goal:** Minimizing latency means making the system *faster* for a single user.
- **Critical for:** Gaming, Voice calls, High-Frequency Trading.

B. Throughput (Capacity)

- **Definition:** The number of actions or data volume the system can handle in a given time period.
- **Measured in:** Requests Per Second (RPS), Transactions Per Second (TPS), or Bits Per Second (bps).
- **Goal:** Maximizing throughput means handling *more users* at once.
- **Critical for:** File uploads, Video streaming, Big Data processing.

The Highway Analogy:

- **Latency** is how fast a car drives (e.g., 100 km/h).
- **Throughput** is how many cars pass a specific point on the highway in one hour.
- *Note:* A wider highway (more bandwidth) increases throughput but does not necessarily decrease latency (the speed limit is still the speed limit).

Part 2: Key Theorems & Consistency Models

In a distributed system (where data is stored across multiple computers), things break. Networks fail, and messages get lost. These theorems govern how we handle those failures.

1. The CAP Theorem

Proposed by Eric Brewer, this theorem states that a distributed system can only provide **two** of the following three guarantees simultaneously.

The Three Guarantees:

1. **Consistency (C):** Every read receives the most recent write or an error. In other words, all nodes see the same data at the same time.
2. **Availability (A):** Every request receives a (non-error) response, without the guarantee that it contains the most recent write. The system is always "up" for writes/reads.
3. **Partition Tolerance (P):** The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network between nodes.

The "Real" Choice: Since networks are unreliable, **Partition Tolerance (P) is mandatory** in any distributed system. You cannot design a system that assumes the network will never fail. Therefore, your actual choice is between **CP** and **AP**.

- **CP (Consistency + Partition Tolerance):**
    - **Behavior:** If communication breaks between nodes, the system shuts down non-consistent nodes or blocks writes to prevent data mismatch.
    - **Trade-off:** You sacrifice Availability. Users might get a "System Error" or timeout.
    - **Use Case:** Banking, Financial systems (you really don't want your balance to be wrong).
    - **Tech:** MongoDB (by default), HBase, Redis.
- **AP (Availability + Partition Tolerance):**
    - **Behavior:** If communication breaks, the system keeps accepting writes on all nodes, even if they can't talk to each other. The nodes will resync data later.

- **Trade-off:** You sacrifice Consistency. Users might see slightly old data (e.g., a "like" count that is off by 5).
- **Use Case:** Social Media feeds, E-commerce catalogs (better to show a product than an error).
- **Tech:** Cassandra, DynamoDB, Couchbase.

---

2. ACID vs. BASE

This is the battle between strict SQL reliability and flexible NoSQL performance.

A. ACID (The SQL Standard) ACID properties guarantee that transactions are processed reliably.

- **Atomicity:** "All or Nothing." If one part of a transaction fails, the entire transaction fails. (e.g., If money leaves A but fails to reach B, the money returns to A).
- **Consistency:** The database moves from one valid state to another valid state. All data validation rules (constraints) are respected.
- **Isolation:** Concurrent transactions result in the same state as if they were executed sequentially. One user's unfinished transaction isn't visible to another.
- **Durability:** Once a transaction is committed, it remains committed even in the case of a system failure (power outage).

B. BASE (The NoSQL Philosophy) BASE is designed for high availability and scalability, often relaxing the strictness of ACID.

- **Basically Available:** The system guarantees availability, but maybe not for *all* operations (e.g., you can view posts, but maybe not comments right now).
- **Soft State:** The state of the system may change over time, even without input, because of eventual consistency (nodes syncing in the background).
- **Eventual Consistency:** If new updates stop, the system will *eventually* propagate all updates to all nodes and become consistent.

---

3. Consistency Patterns (The Spectrum)

Consistency isn't just binary (Strong vs. Weak). There is a spectrum:

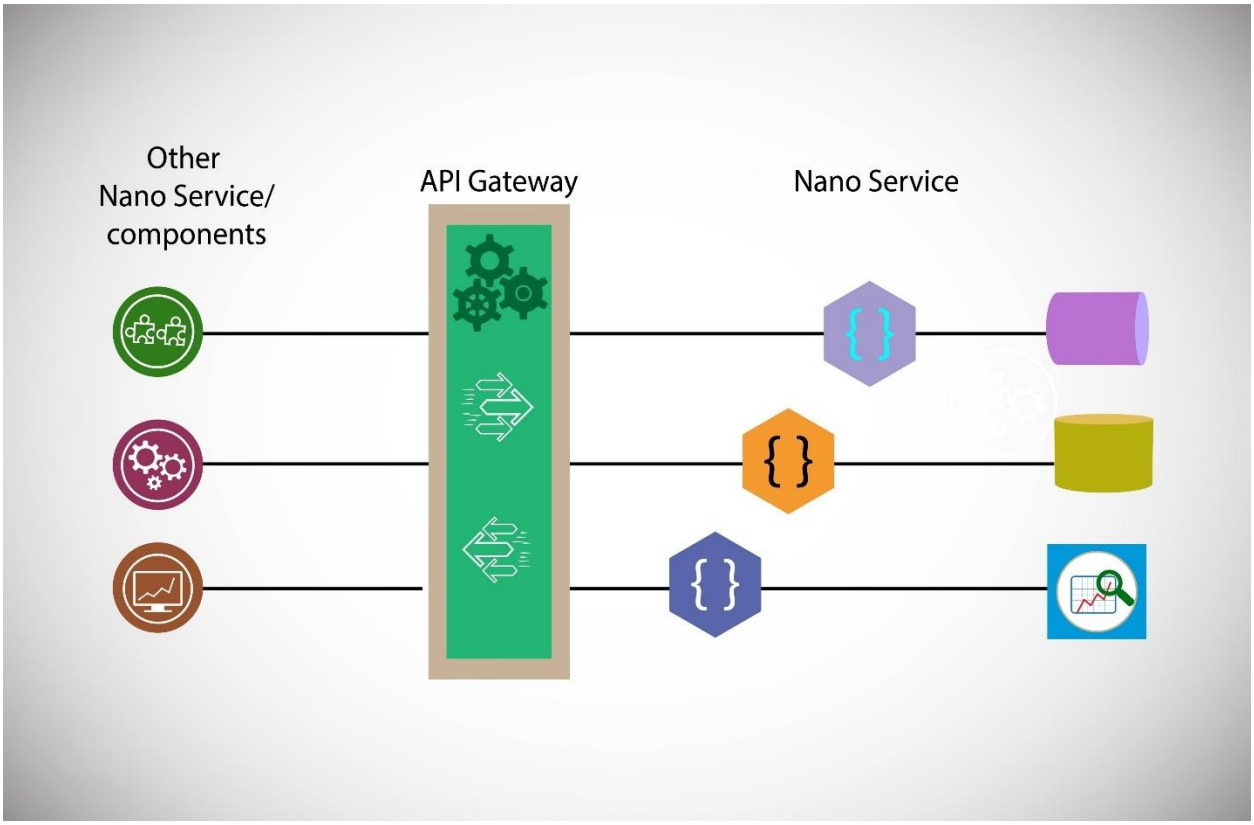
- **Strong Consistency:** After a write, any subsequent read *immediately* sees that value. (Highest latency, highest data integrity).
- **Weak Consistency:** After a write, reads may or may not see the value. Used in caching (VoIP, video chat).
- **Eventual Consistency:** The specific form of weak consistency used in DNS and email. "It will show up... soon."

Part 3: Core Components

These are the lego bricks of system design. You need to know when to use them and, more importantly, *why*.

1. Load Balancers (LB)

A Load Balancer stands in front of your servers and acts as a "traffic cop," distributing incoming client requests across multiple servers. This ensures no single server is overwhelmed.



Shutterstock

Key Concepts:

- **Layer 4 vs. Layer 7 Balancing:**
    - **Layer 4 (Transport Layer):** Routes traffic based on IP address and Port (TCP/UDP). It doesn't look inside the message. It is very fast but "dumb."
    - **Layer 7 (Application Layer):** Routes traffic based on the content (URL, Headers, Cookies). Can route /video to video servers and /chat to chat servers. It is smarter but slightly slower (CPU intensive).
- **Routing Algorithms:**
    - **Round Robin:** Sequential (Server A -> Server B -> Server C -> A...). Good for servers with equal specs.
    - **Least Connections:** Sends traffic to the server with the fewest active connections. Best for long sessions (e.g., chat).
    - **IP Hash:** Hashes the client's IP to ensure a specific user always goes to the same server ("Sticky Sessions").

---

2. Caching

Caching is the strategy of storing copies of frequently accessed data in fast temporary storage (usually RAM) to reduce latency and load on the database.

Where does it live?

1. **Client-Side:** Browser cache (cookies, local storage).
2. **CDN (Content Delivery Network):** Caches static assets (images, CSS, video) on servers geographically closer to the user (e.g., Cloudflare, Akamai).
3. **Application Server:** In-memory cache (local variable).
4. **Distributed Cache:** A dedicated caching layer shared by all servers (Redis, Memcached).

Caching Strategies (Crucial for Interviews):

- **Cache-Aside (Lazy Loading):** The application looks in the cache first. If it's missing (cache miss), it loads from the DB and updates the cache.
    - *Pro:* Cache only contains requested data.
    - *Con:* First request is slow.

- **Write-Through:** Data is written to the cache and the DB at the same time.
    - *Pro:* High data consistency; data is safe.
    - *Con:* Slower writes (two operations).
- **Write-Back (Write-Behind):** Data is written only to the cache initially. The cache asynchronously writes to the DB later.
    - *Pro:* Extremely fast writes.
    - *Con:* Risk of data loss if the cache crashes before syncing to the DB.

---

3. Databases: SQL vs. NoSQL

Choosing the right storage is often 50% of the interview score.

A. Relational (SQL)

- **Examples:** MySQL, PostgreSQL, Oracle.
- **Structure:** Tables with rows and columns. Pre-defined schema.
- **Best For:** Complex relationships, transactions (Banking), rigid data structures.
- **Scaling:** Great at Vertical scaling; harder to scale Horizontally.

B. Non-Relational (NoSQL)

- **Key-Value:** (Redis, DynamoDB) - Extremely fast. Good for caching, session management.
- **Document:** (MongoDB) - JSON-like documents. Flexible schema. Good for CMS, catalogs.
- **Column-Family:** (Cassandra, HBase) - Optimized for massive writes and reads. Good for time-series data, logs.
- **Graph:** (Neo4j) - Stores relationships as first-class citizens. Good for social networks, recommendation engines.

---

4. Database Scaling

When your database gets too big or too slow, you have two main moves:

A. Replication (Read Scaling) You create copies of your database.

- **Master-Slave:** All writes go to the "Master". The Master copies data to "Slaves". All reads come from Slaves.
    - *Issue:* If Master dies, a Slave must be promoted (downtime/complexity).
- **Master-Master:** Writes can go to any node.
    - *Issue:* Handling write conflicts is very difficult.

B. Sharding (Write Scaling) You split your data across multiple machines based on a "Shard Key" (e.g., UserID).

- *Example:* Users 1-1000 go to DB Server A; Users 1001-2000 go to DB Server B.
- *Pro:* Infinite write scaling.
- *Con:* Complex queries (Joins across shards are painful/impossible).

---

5. Message Queues (MQ)

MQs allow different parts of your system to communicate asynchronously. They "decouple" services.

- **Examples:** Kafka, RabbitMQ, Amazon SQS.
- **Scenario:** A user uploads a video.
    - *Without MQ:* The user waits 5 minutes for the video to process. (Bad UX).
    - *With MQ:* The web server puts a "Process Video" message in the Queue and immediately tells the user "Upload Successful!" A background worker picks up the message and processes the video later.
- **Benefit:** Buffer against traffic spikes (Load Leveling).

Part 4: Communication Protocols & Critical Patterns

1. Communication Protocols

How do your services exchange data? You usually choose between standard HTTP (REST), high-performance RPC, or real-time WebSockets.

A. REST (Representational State Transfer)

- **Concept:** The standard for web APIs. Everything is a "resource" accessed via standard HTTP verbs (GET, POST, PUT, DELETE).
- **Format:** Typically JSON.
- **Pros:** Universal support, easy to debug (human-readable), stateless.
- **Cons:** "Chatty" (lots of metadata/headers in every request), JSON serialization is slower than binary.
- **Use Case:** Public-facing APIs (e.g., Stripe, Twitter).

B. RPC (Remote Procedure Call) & gRPC

- **Concept:** A client calls a function on a server as if it were a local function: service.getUser(id).
- **Format:** Typically **Protobufs** (Protocol Buffers) via **gRPC** (Google RPC). This compresses data into binary, making it much smaller than JSON.
- **Pros:** Extremely fast, lightweight, type-safe (strict contracts).
- **Cons:** Harder to debug (binary is not human-readable), requires browser support libraries (not native like REST).
- **Use Case:** Internal microservices communication (Service A talking to Service B).

C. WebSockets

- **Concept:** A persistent, two-way (full-duplex) communication channel over a single TCP connection.
- **Pros:** Real-time data push. The server can send data to the client without the client asking for it.
- **Cons:** Requires maintaining a persistent connection (stateful), which is harder to scale than stateless REST.
- **Use Case:** Chat apps, Live sports scores, Multiplayer gaming.

---

2. Critical System Patterns

These are specific "safety mechanisms" you design to keep your system from crashing under pressure.

A. Rate Limiting

- **Definition:** Controlling the rate of traffic sent or received by a network interface controller.
- **Why:** To prevent DoS (Denial of Service) attacks and ensure one heavy user doesn't starve others of resources.
- **Where:** Implemented at the API Gateway or Load Balancer level.
- **Common Algorithms:**
    - **Token Bucket:** You get tokens at a fixed rate. Each request costs a token. If the bucket is empty, the request is dropped. Allows for "bursts" of traffic.
    - **Leaky Bucket:** Requests enter a queue and are processed at a constant fixed rate. Smooths out traffic spikes completely.

B. Circuit Breaker

- **Problem:** If Service A depends on Service B, and Service B fails (hangs), Service A will keep waiting, consuming threads/resources until it also crashes (Cascading Failure).
- **Solution:** The Circuit Breaker wraps the function call.
    - **Closed State:** Normal operation. Requests go through.
    - **Open State:** If failure rate exceeds a threshold (e.g., 50%), the "circuit opens." All requests fail immediately *without* calling the downstream service. This gives the failing service time to recover.
    - **Half-Open State:** After a timeout, let a *few* requests through to test if the service is back up. If yes, Close circuit. If no, Open again.

C. Consistent Hashing

- **The Problem:** In standard load balancing (hash(userID) % N), if you add or remove a server (changing N), *almost all* keys get remapped to different servers. This kills your cache.

- **The Solution:** Map both the Servers and the Requests onto a circular ring (0 to 360 degrees).
- **Mechanism:**
    1. Hash servers to points on the ring.
    2. Hash the Request key to the ring.
    3. Move clockwise to find the first server. That server handles the request.
- **Benefit:** Adding/removing a server only affects the keys between it and its neighbor. Only ~1/N keys need to be moved.
- **Virtual Nodes:** To prevent "hotspots" (one server getting too much data), we hash each server multiple times (Server A1, A2, A3...) to spread them evenly around the ring.

==========

1. The Core Problem: Why HTTPS?

Standard HTTP sends data in **plaintext**. If you log in to a bank over HTTP, anyone on the Wi-Fi network can "sniff" the packets and read your password.

HTTPS (HTTP Secure) fixes this by wrapping HTTP inside **TLS (Transport Layer Security)**. It guarantees three things:

1. **Encryption:** Only the sender and receiver can read the data.
2. **Integrity:** The data was not modified in transit.
3. **Authentication:** You are actually talking to the server you think you are (e.g., google.com, not a hacker acting like Google).

---

2. The Building Blocks: Symmetric vs. Asymmetric

To understand the handshake, you must understand the two types of cryptography used.

A. Symmetric Encryption (The "Fast" Way)

- **Concept:** The **same key** is used to lock (encrypt) and unlock (decrypt) the data.
- **Analogy:** A house key. You use the same key to lock the door when leaving and unlock it when returning.

- **Pros:** Extremely fast and efficient for large amounts of data.
- **Cons: The Distribution Problem.** How do I send you the key securely? If I email it to you, a hacker can steal it.

B. Asymmetric Encryption (The "Safe" Way)

- **Concept:** There is a **pair** of mathematically linked keys:
    1. **Public Key:** Everyone can see it. Used to **Encrypt**.
    2. **Private Key:** Only the owner has it. Used to **Decrypt**.
- **Analogy:** A mailbox. Anyone can drop a letter in (Public Key), but only the postman with the master key can open it and take letters out (Private Key).
- **Pros:** No need to share the secret key.
- **Cons:** Very slow and computationally heavy.

The Hybrid Solution (TLS): We use the *slow* Asymmetric encryption **only once** to safely exchange a *fast* Symmetric key. Then, we use the Symmetric key for the rest of the conversation.

---

3. The TLS Handshake (The "Dance")

This is the step-by-step process that happens in milliseconds before a webpage loads. We will focus on TLS 1.2/1.3 (the modern standards).

Step 1: Client Hello

- The browser (Client) sends a message to the Server.
- **Content:** "I support TLS 1.2 and 1.3. I support these encryption algorithms (Cipher Suites). Here is a random number (Client Random)."

Step 2: Server Hello

- The Server picks the best encryption algorithm from the list.
- **Content:** "Let's use TLS 1.3. Here is my SSL Certificate (containing my **Public Key**). Here is my random number (Server Random)."

Step 3: Verification (Authentication)

- The Browser checks the Certificate with a **Certificate Authority (CA)** (like Let's Encrypt or DigiCert) to ensure it's valid and actually belongs to google.com.
- *If invalid:* You get that big scary red warning in Chrome ("Your connection is not private").

Step 4: Key Exchange (The Magic)

- **The Pre-Master Secret:** The Client generates a new random string called the "Pre-Master Secret."
- **Encryption:** The Client encrypts this secret using the Server's **Public Key** (from the certificate).
- **Transmission:** The Client sends this encrypted blob to the Server.
- **Decryption:** The Server uses its **Private Key** to decrypt it. Now, *both* sides have the Pre-Master Secret, and no one else (not even the ISP) could have seen it.

Step 5: Session Keys

- Both the Client and Server use the *Client Random*, *Server Random*, and *Pre-Master Secret* to mathematically generate the exact same **Session Key** (Symmetric Key).

Step 6: Secure Communication

- The Handshake is over. All future HTTP data is encrypted using the fast **Session Key**.

---

4. System Design Implication: SSL Termination

Because the handshake (specifically the Asymmetric part) requires heavy math, it consumes CPU. If your app server is handling thousands of handshakes *and* running business logic, it might slow down.

The Solution: SSL Termination (Offloading) You perform the decryption at the **Load Balancer** or **Reverse Proxy** (e.g., Nginx), not the Application Server.

1. **Traffic (HTTPS):** Encrypted traffic hits the Load Balancer.
2. **Termination:** The Load Balancer has the SSL Certificate. It decrypts the traffic.
3. **Traffic (HTTP):** The Load Balancer sends plain HTTP traffic to your internal Application Servers (inside your private secure network).
4. **Benefit:** Your App Servers save CPU cycles for actual work (generating HTML/JSON).

Summary for Interviews

1. **Hybrid Approach:** Asymmetric to share the key; Symmetric to transfer the data.
2. **Certificates:** Establish trust via Certificate Authorities (CAs).
3. **Handshake:** The process of agreeing on the keys.
4. **Termination:** Decrypt at the Load Balancer to save App Server resources.

Tiny URL:

Here is the system design for a **URL Shortener** (like TinyURL or Bitly). This is one of the most common system design interview questions because it seems simple but requires solving a specific algorithmic challenge: *Generating a unique, collision-free short key.*

---

1. Requirements Clarification

Always start by defining the scope.

Functional Requirements:

1. **Shorten:** Given a long URL, generate a shorter, unique alias.
2. **Redirect:** When a user accesses the short link, redirect them to the original long URL.
3. **High Availability:** The system cannot go down; redirection must happen instantly.

Non-Functional Requirements:

1. **Read-Heavy:** Many more people click links than create them (Assume 100:1 ratio).
2. **Low Latency:** Redirection should happen in milliseconds.
3. **Non-predictable:** (Optional but good) Short keys shouldn't be sequential (attacker shouldn't be able to guess the next URL).

---

2. Back-of-the-Envelope Estimation

Show the interviewer you can do "napkin math."

- **Traffic:**
    - Assume **100 Million** new URLs generated per month.
    - **Writes (QPS):** $\frac{100,000,000}{30 \times 24 \times 3600} \approx 40$ writes/sec. Let's round to **100 writes/sec**.
    - **Reads (QPS):** 100:1 ratio $\rightarrow$ **10,000 reads/sec**.
- **Storage:**
    - Assume 500 bytes per entry (Long URL + Metadata).
    - 100M entries/month $\times$ 500 bytes = 50 GB/month.
    - **5 Years:** $50 \text{ GB} \times 12 \times 5 = 3 \text{ TB}$.
    - *Conclusion:* 3TB is manageable. A NoSQL database is a good fit.

---

3. API Design

A. Create Short URL

- POST /api/shorten
- **Body:** { "long_url": "https://www.google.com/..." }
- **Return:** 200 OK, { "short_url": "http://tiny.url/jK7l9" }

B. Redirect (The Trick)

- GET /{short_key}
- **Return:** 301 Permanent Redirect vs. 302 Found (Temporary Redirect).
    - **301 (Permanent):** The browser caches the redirect. Future requests go *directly* to the long URL, skipping your server.
        - *Pro:* Reduces load on your server.
        - *Con:* You lose analytics (you can't track how many times the link was clicked).
    - **302 (Temporary):** The browser hits your server *every time*.
        - *Pro:* Accurate analytics.
        - *Con:* Higher server load.

- *Verdict:* For a system design interview, usually choose **302** because analytics are business-critical for companies like Bitly.

---

4. Database Schema

We need a table to map the Short Key to the Long URL.

Since we need massive scale and fast lookups (key-value), a NoSQL store like DynamoDB, Cassandra, or Riak is better than SQL.

Table: UrlMapping

| Column | Type | Description |

| :--- | :--- | :--- |

| Hash (PK) | String | The short key (e.g., jK7l9) |

| OriginalUrl | String | The long URL |

| CreationDate | Date | For expiration logic |

| UserId | String | (Optional) Who created it |

---

5. The Core Problem: Generating the Key

This is the heart of the interview. How do we turn a long input into a 7-character unique string?

Approach A: Hashing (The Trap)

You hash the Long URL using MD5 or SHA256.

- *MD5(long_url)* = e4d909c290d0fb1ca068ffaddf22cbd0 (too long).
- You take the first 7 characters: e4d909c.
- **The Problem (Collisions):** Two different long URLs might accidentally produce the same first 7 characters. Handling these collisions is messy.

Approach B: Base62 Conversion (The Solution)

This is the standard industry approach.

1. **Concept:** Convert a unique **Integer ID** (Database ID) into a **Base62 number**.

2. **Why Base62?** It uses [a-z, A-Z, 0-9], which is $26 + 26 + 10 = 62$ characters.
3. **Capacity:** With 7 characters in Base62 (62^7), we can generate **3.5 Trillion** unique URLs. This is enough for 100 years.

The Algorithm:

- ID 1000 $\rightarrow$ Base62 conversion $\rightarrow$ g8
- ID 1000000 $\rightarrow$ Base62 conversion $\rightarrow$ 4c92
- ID 12345678901 $\rightarrow$ Base62 conversion $\rightarrow$ Tx91bN

But how do we get a unique Integer ID in a distributed system?

If we use a standard database Auto-Increment, the database becomes a bottleneck.

Solution: Key Generation Service (KGS)

We build a standalone service that pre-generates unique keys and stores them in a separate database or loads them into memory.

- **Worker:** Takes a generic range (e.g., 1 to 1,000,000).
- **App Server:** Asks KGS, "Give me a key."
- **KGS:** Returns jK7l9 and marks it as "used."
- *Benefit:* Zero collisions, extremely fast.

---

6. Final Architecture

1. **User** sends create request.
2. **Load Balancer** forwards to Web Server.
3. **Web Server** requests a unique ID from the **Key Generation Service (KGS)**.
4. **KGS** provides a unique ID (e.g., 784321), converts it to Base62 (Xy7z), and returns it.
5. **Web Server** stores { "Xy7z": "https://google.com..." } in the **NoSQL DB**.
6. **Web Server** returns tiny.url/Xy7z to the user.

For Redirects (Reads):

1. **User** visits tiny.url/Xy7z.
2. **Load Balancer** hits Web Server.

3. **Web Server** checks **Redis Cache**.
    - *Hit:* Return 302 Redirect immediately.
    - *Miss:* Fetch from DB, update Cache, return 302 Redirect.

---

7. Advanced Bottlenecks (Bonus Points)

- **Cache Eviction:** Use **LRU (Least Recently Used)**. If the cache is full, remove the link that hasn't been clicked in the longest time.
- **Database Cleanup:** Links often expire (e.g., 5 years). Don't scan the whole DB to delete them (too slow). instead, use "Lazy Cleanup":
    - If a user tries to access an expired link, delete it then and return error.
    - Run a lightweight background script during off-peak hours to remove old entries.

EXAMPLE FOR SYSTEM DESIGN

This is a **Senior Engineer/Architect level** breakdown. To make this "highly technical," we move past simple database auto-increments and solve for **distributed concurrency**, **latency**, and **massive scale**.

1. The Distributed Architecture

At a high level, we cannot rely on a single database to generate IDs (Single Point of Failure). We need a **Key Generation Service (KGS)**.

The Data Flow:

1. **Write:** Client $\to$ Load Balancer $\to$ App Server $\to$ **KGS** (Get Unique ID) $\to$ **Base62 Encode** $\to$ DB (Write Async) $\to$ Cache $\to$ Return Short URL.
2. **Read:** Client $\to$ Load Balancer $\to$ App Server $\to$ **Bloom Filter** (Check existence) $\to$ **Redis Cache** (Hit?) $\to$ DB (Miss?) $\to$ 302 Redirect.

---

2. The "Secret Sauce": Distributed ID Generation

We will use a **Token Range Strategy** managed by **Apache Zookeeper** (or etcd).

- **The Problem:** Generating a unique ID across 50 different servers without collisions and without locking the database.
- **The Solution:**
    1. **Zookeeper** acts as the "Range Manager." It keeps track of the global counter (e.g., currently at 1,000,000).
    2. **App Server A** boots up. It asks Zookeeper for a range.
    3. Zookeeper assigns 1,000,001 to 1,001,000 to Server A and updates the global counter.
    4. **App Server A** now generates IDs in memory (AtomicInteger) from this range.
    - **Speed:** Zero network calls per URL generation. It's purely in-memory math.
    - **Safety:** Zookeeper guarantees no two servers get the same range.

---

3. The Code Implementation (Java)

Here is the implementation of the core logic: **Base62 Conversion** and the **Service Layer** handling the logic.

A. The Base62 Encoder (Math Utility)

We map a Base 10 integer (from our ID generator) to a Base 62 string.

$$Key Space = 62^7 \approx 3.5 \text{ Trillion combinations}$$

Java

```
import java.util.Optional;


public class Base62 {
   private static final String ALPHABET =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
   private static final int BASE = ALPHABET.length(); // 62


   // Encode: Turns ID 1000 -> "qi"
   public static String encode(long id) {
      StringBuilder sb = new StringBuilder();
      if (id == 0) return String.valueOf(ALPHABET.charAt(0));


      while (id > 0) {
         sb.append(ALPHABET.charAt((int) (id % BASE)));
         id /= BASE;
      }
      return sb.reverse().toString();
   }


   // Decode: Turns "qi" -> 1000 (Useful for analytics/debugging)
   public static long decode(String str) {
```

```
        long id = 0;
        for (int i = 0; i < str.length(); i++) {
            id = id * BASE + ALPHABET.indexOf(str.charAt(i));
        }
        return id;
    }
}
```

B. The Key Generation Service (Token Range Simulation)

In a real system, fetchNewRangeFromZookeeper() would make a network call. Here, I simulate the in-memory atomic efficiency.

Java

```
import java.util.concurrent.atomic.AtomicLong;

public class KeyGenerationService {
    // Current range assigned to this specific server instance
    private AtomicLong currentId;
    private long maxIdInRange;

    // Zookeeper Client (Mocked)
    private ZookeeperClient zkClient;

    public KeyGenerationService() {
        // On startup, fetch the first range
        refreshRange();
    }
```

```
    private synchronized void refreshRange() {
        // Simulates calling Zookeeper to get a range like [10000, 20000]
        Range range = zkClient.getAvailableRange();
        this.currentId = new AtomicLong(range.getStart());
        this.maxIdInRange = range.getEnd();
    }

    public long generateUniqueId() {
        long id = currentId.getAndIncrement();

        // If we exhausted our range, get a new one immediately
        if (id > maxIdInRange) {
            refreshRange();
            // Recursive call to get ID from new range
            return generateUniqueId();
        }
        return id;
    }
}
```

C. The Controller / Service Layer

This ties it all together with caching strategies.

Java

```
@Service
public class UrlShortenerService {

    @Autowired private KeyGenerationService keyGen;
```

```
@Autowired private RedisTemplate<String, String> redisCache;
@Autowired private CassandraRepository db;

// WRITE PATH
public String shortenUrl(String longUrl) {
   // 1. Get Unique ID (In-Memory, incredibly fast)
   long id = keyGen.generateUniqueId();

   // 2. Base62 Encode
   String shortKey = Base62.encode(id);

   // 3. Save to DB (Async recommended for speed)
   // Schema: [Key: shortKey, Value: longUrl, CreatedAt: timestamp]
   db.save(new UrlMapping(shortKey, longUrl));

   // 4. Update Cache (Write-Through)
   redisCache.opsForValue().set(shortKey, longUrl);

   return "https://tiny.url/" + shortKey;
}

// READ PATH
public String getLongUrl(String shortKey) {
   // 1. Check Cache
   String cachedUrl = redisCache.opsForValue().get(shortKey);
   if (cachedUrl != null) {
```

```
        return cachedUrl;
    }

    // 2. Bloom Filter Check (Optimization)
    // If Bloom Filter says "No", we skip DB. Prevents DB spamming.
    if (!bloomFilter.mightContain(shortKey)) {
        throw new NotFoundException();
    }

    // 3. DB Fallback
    UrlMapping mapping = db.findById(shortKey);

    // 4. Populate Cache (Cache-Aside)
    if (mapping != null) {
        redisCache.opsForValue().set(shortKey, mapping.getLongUrl());
        return mapping.getLongUrl();
    }

    return null; // 404
  }
}
```

---

4. Storage Layer: Why Cassandra?

For a URL shortener, we want **massive write throughput** and **easy scalability**.

Schema (Cassandra CQL):

SQL

```
CREATE TABLE url_mapping (
  short_key text PRIMARY KEY,
  long_url text,
  created_at timestamp
);
```

Why not MySQL?

MySQL is great, but eventually, you will hit the limit of a single master node. You would have to implement manual sharding (e.g., Shard 1: keys a-m, Shard 2: keys n-z).

Cassandra handles sharding automatically via Consistent Hashing. It provides extremely fast writes because it writes to a commit log and memory (MemTable) before flushing to disk.

---

5. "Pro" Optimizations (Technical Deep Dive)

If you want to impress the interviewer, mention these specific technical constraints:

1. **Bloom Filters:**
    - *Scenario:* A hacker writes a script to request random keys (tiny.url/abc, tiny.url/xyz) to overload your DB.
    - *Solution:* A Bloom Filter is a probabilistic data structure that sits in memory. It tells you if a key **definitely does not exist**. If the Bloom Filter says "No," you return 404 immediately without touching the Redis or Database.
2. **301 vs. 302 Redirects:**
    - In the code, you must decide the HTTP status.
    - **301 (Permanent):** Browser caches the mapping. Fast for user, but you **lose analytics** (server never sees the request again).
    - **302 (Temporary):** Request always hits your server. Slightly slower, but allows you to track click stats. **Choose 302 for this system.**
3. **Cleanup (TTL):**
    - Do not run a "cron job" to delete expired rows (it kills DB performance).

- Use Cassandra's native **TTL (Time To Live)** feature.
- INSERT INTO url_mapping ... USING TTL 157680000 (5 years). The DB automatically deletes the data when time expires.

ADVANCED:

Part 5: Advanced Data Consistency (Microservices)

When you break a monolith into microservices, you lose ACID transactions across tables. If Service A charges a user and Service B ships the product, how do you ensure both happen, or neither happens?

1. Two-Phase Commit (2PC)

The "Old School" Strict approach.

- **Phase 1 (Prepare):** The Coordinator asks all services: "Can you commit this transaction?" Services lock their data and say "Yes" or "No."
- **Phase 2 (Commit):** If *everyone* said Yes, the Coordinator says "Commit." If *anyone* said No, the Coordinator says "Rollback."
- **Problem:** It is **blocking**. If the Coordinator crashes, all services are stuck waiting with locks held. It doesn't scale well.

2. The Saga Pattern (The Standard)

The "Modern" Asynchronous approach. Instead of one big transaction, you break it into a sequence of local transactions.

- **Flow:** Service A completes task $\rightarrow$ Events $\rightarrow$ Service B completes task.
- **Failure Handling (Compensating Transactions):** If Service B fails, you must trigger a "undo" logic for Service A.
    - *Example:*
        1. OrderService: Create Order (Success) $\rightarrow$ Trigger Payment.
        2. PaymentService: Charge Card (Failed: Insufficient Funds).
        3. PaymentService: Send "Fail Event".
        4. OrderService: Listens to event $\rightarrow$ Execute CancelOrder() (The Compensating Transaction).

Part 6: Search & Discovery Systems

Standard Databases (SQL/NoSQL) are terrible at searching for text (e.g., "Find all tweets containing 'System Design'").

1. Inverted Index (Elasticsearch / Lucene)

This is the data structure behind Google search, Amazon product search, and logging (ELK stack).

- **Concept:** Instead of mapping Document $\rightarrow$ Words, it maps Words $\rightarrow$ Documents.
    - *Doc 1:* "Blue Sky"
    - *Doc 2:* "Blue Ocean"
- **The Index:**
    - Blue $\rightarrow$ [Doc 1, Doc 2]
    - Sky $\rightarrow$ [Doc 1]
    - Ocean $\rightarrow$ [Doc 2]
- **Query:** Search for "Blue" returns the list [1, 2] instantly (O(1) lookup).

2. Typeahead / Autocomplete (Tries)

How does Google suggest searches as you type?

- **Data Structure: Trie** (Prefix Tree).
- **Mechanism:**
    - Root $\rightarrow$ 'd' $\rightarrow$ 'e' $\rightarrow$'s' $\rightarrow$ 'i' $\rightarrow$ 'g' $\rightarrow$ 'n'.
    - At the node 'n', we store the top 5 most popular searches starting with "design" (e.g., "design patterns", "designer").
- **Optimization:** You cannot traverse the tree every time. You cache the top 5 results at *every* node of the tree.

Part 7: The "Feed" Architecture (Fan-Out)

This is the core design for **Twitter, Instagram, and Facebook News Feeds**. The challenge is distributing one user's post to millions of followers.

1. Pull Model (Fan-out on Read)

- **How it works:** When you create a post, it just saves to the DB. When a follower opens their feed, the system queries all the people they follow, merges the lists, sorts by time, and returns the feed.
- **Pros:** Very fast writes.
- **Cons:** Very slow reads. If you follow 1,000 people, the system has to query 1,000 tables every time you refresh.

2. Push Model (Fan-out on Write)

- **How it works:** When you create a post, the system *immediately* pushes that post ID into a pre-computed "Feed List" for every single one of your followers.
- **Pros:** Instant reads. The feed is already built; you just fetch it.
- **Cons:** "The Justin Bieber Problem." If a user has 100 million followers, writing one tweet triggers 100 million updates. This causes "Thundering Herd" latency.

3. Hybrid Approach (The Solution)

- **Normal Users:** Use **Push**. Push their tweets to followers.
- **Celebrities:** Use **Pull**. Don't push their tweets. When a user loads their feed, fetch normal tweets (Push cache) + mix in celebrity tweets (Pull DB).

Part 8: Geo-Spatial Systems (Uber/Yelp)

How do you find "drivers near me"? You cannot scan the whole database of locations.

1. QuadTree

- **Concept:** Divide the world map into 4 squares (quadrants). If a square has too many points (drivers), divide it into 4 smaller squares. Repeat recursively.
- **Search:** "Find drivers in this box." The tree quickly eliminates 99% of the world.
- **Issue:** If the map changes frequently (drivers moving), updating the tree is expensive.

2. Geohash (String-based)

- **Concept:** Convert 2D coordinates (Lat/Long) into a 1D string.

- **Mechanism:** Divide the world into a grid. Each grid has a binary string ID.
    - London might be gcpv.
    - A specific street in London might be gcpvj.
- **The Trick:** Strings that share a long prefix are geographically close. To find nearby drivers, you just search the DB for users whose Geohash starts with gcpv....

Part 9: Large Scale File Storage (Netflix/Dropbox)

How do you store Petabytes of videos or images? You don't use a database. You use **Blob Storage** (like AWS S3).

Architecture

1. **Metadata DB:** Stores the "pointer" (filename, uploader ID, upload date, file path). Fast lookup.
2. **Blob Store:** Stores the actual binary object.
    - **Chunking:** Large files (movies) are split into small chunks (e.g., 64MB).
    - **De-duplication:** If two people upload the same movie, you only store the chunks once.

Summary Checklist for "Completeness"

If you walk into an interview, you now have the tools for any archetype:

Requirement	**Tool / Pattern to Use**
"Fast lookups"	Distributed Cache (Redis) + Consistent Hashing
"Relationships"	Graph DB (Neo4j) or SQL
"Search Text"	Inverted Index (Elasticsearch)
"Heavy processing"	Message Queues (Kafka)
"Real-time updates"	WebSockets

Requirement	Tool / Pattern to Use
"Global Scale"	CDN + Geo-DNS
"Nearby location"	Geohash / QuadTree
"Microservices Logic"	Saga Pattern
"Top K Elements"	Count-Min Sketch or Heap
"Unique IDs"	Zookeeper Range or Snowflake ID